

Numbers

Python Numbers probably look the simplest and might not seem like an advanced object, but considering the vast variety of numbers that python can accommodate out of the box, it certainly lives up to the name. The Number class hierarchy in python closely resembles a tree, as the Number class branches into various sub-classes that progressively represent a subset of the Numbers and define more operations specific to this subset.

The Numbers module supports almost the entire universe of Numbers that include integers, floating-point values, exponential numbers and even complex numbers. It also provides support for binary, octal and hexadecimal numbers.

A number can belong to one of the three types that are int, float, and complex representing integers, floating point numbers and complex numbers respectively. In order to find the type of a number, we can use the `type()` method. It prints the class to which the supplied number belongs.

```
>>> a = 5
>>> type(a)
<class 'int'>
>>> b = 2.15
>>> type(b)
<class 'float'>
```

Any number in python which contains a fractional part is always `float`, even if the fractional part is 0.

```
>>> d = 4.00
>>> type(d)
<class 'float'>
```

A complex number in python is of the form (`real+imag*j`). The imaginary part should always have a coefficient and it should always be represented by the character 'j'. A number will be considered as a complex number even if the imaginary coefficient supplied to the number is 0.

```
>>> c = 3 + 2j
>>> type(c)
<class 'complex'>
```

Arithmetic Operations

Since we are dealing with numbers, at some point we would need to perform arithmetic operations on them. The Numbers module allows operations between all numeric types with an exception of few restricted operations.